

Lecture 1: NP-Completeness

*Lecturer: Tanner Duve, Toyosi Abu**Scribes: N/A*

1.1 Recap

The class $\mathcal{P}$: Set of “easy” problems. Problems with an algorithm that solves it in n^k steps where n is the size of the input and $k \in \mathbb{N}$ (polynomial time)

The class $\mathcal{NP}$: Set of problems that can be verified quickly, ie. given an instance of a problem and a “certificate”, a verifier can check if the certificate is a valid solution to the problem in polynomial time.

Polynomial time reduction: A function which “transforms” instances of X to instances of Y in polynomial time, ie. a function f such that $x \in X \iff f(x) \in Y$. We say X is polynomial time reducible to Y (denote $X \leq_p Y$) iff such a function exists.

$\mathcal{NP}$ -complete: These are the “maximally hard” problems in NP. X is NP-complete iff every problem in NP reduces to it.

Theorem 1.1 (*Cook-Levin*) *SAT is NP-Complete*

Corollary 1.2 $SAT \in \mathcal{P} \iff \mathcal{P} = \mathcal{NP}$

Theorem 1.3 *If X is NP-Complete, and $X \leq_p Y$, then Y is NP-complete*

Proof: Suppose X is NP-complete and that $X \leq_p Y$. By definition we know that every problem in NP reduces to X . For any $X' \in \mathcal{NP}$, compose the reduction from X' to X with the reduction from X to Y to get a reduction from X' to Y . ■

1.2 3-Satisfiability, AKA 3SAT

3SAT is a special case of SAT where each clause contains exactly 3 literals. An example 3SAT formula is the following:

$$(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1, x_4, x_5) \wedge (x_6 \vee \neg x_4 \vee x_7)$$

We omit the proof here, but we state that 3SAT is NP-complete, by the fact that any SAT formula can be converted into an equivalent 3SAT formula (exercise...) we will use 3SAT for our reductions moving forward.

1.3 Graph Theory Basics

Graphs are a mathematical structure used to model pairwise relationships. Formally a graph G is a pair (V, E) where V is a set of “vertices” and E is a binary relation over V , each pair in E is called an “edge”.

We can visualize graphs diagrammatically, by drawing each element of V as a vertex, and drawing an edge between any pair of vertices in E . For example, the graph $V = \{1, 2, 3\}, E = \{\langle 1, 2 \rangle, \langle 2, 3 \rangle\}$ can be drawn as follows:

(insert image)

Graphs can model all kinds of binary relations, but for the sake of this presentation, in relevance to algorithmic fairness, we use graphs to model social networks, where vertices represent people, and there is an edge between two people if they are friends. In the ex. above, we have 3 people 1, 2, 3, and we have that 1 and 2 are friends, 2 and 3 are friends, but 1 and 3 are not friends.

1.4 Independent Set

Consider the following problem: Given a social network and an integer k , is there a set of k people such that no two people in the set are friends? Such a set is called an independent set of size k . We refer to the problem of answering this question as Independent Set, or Ind-Set for short.

Theorem 1.4 *Independent-Set is NP-complete*

Proof: First we show $IS \in \mathcal{NP}$. This is clear, given a set of k people as a certificate, we can quickly check by querying every pair if no two of them are friends (about k^2 queries).

Finally, we must show NP-completeness. We do this by reduction from 3SAT.

Given a 3-CNF formula $\varphi(x_1, \dots, x_n)$ with clauses $C_1, \dots, C_m$, our goal is to produce a graph G and an integer k such that φ is satisfiable iff G has an independent set of size k .

The reduction proceeds as follows:

Let $\varphi(x_1, \dots, x_n)$ be a 3-CNF boolean formula with m clauses $C_1, \dots, C_m$

We construct a graph G such that for each literal $x_i \in \varphi$, we let x_i be a vertex in G .

For any $x_i, x_j \in \varphi$, if x_i, x_j appear in the same clause, we add an edge between the corresponding vertices in G . Furthermore, for any $x_i \in \varphi$, if $\neg x_i \in \varphi$, we add an edge $\langle x_i, \neg x_i \rangle$ to E .

Return G, m

Claim 1.5 φ is satisfiable iff the resulting graph G has an independent set of size m

Proof: Suppose φ is satisfiable, then there exists a truth assignment which makes it evaluate to true. Since φ is a “and” of clauses, then it must be that this assignment sets each clause to true. Since each clause is an “or” of literals, it must also be that this assignment sets at least one literal in each clause to be true. Select one true literal from each clause and observe that the corresponding vertices form an independent set of size m (check!)

Conversely suppose G has an independent set of size m . This set must contain exactly one vertex from each triangle/clause. Set the corresponding literals to true and we get a satisfying assignment.

To check that this reduction is polynomial is trivial. We iterate through n literals and add at most n^2 edges.

Conclude that $3SAT \leq_p \text{IND-SET}$, so by theorem 1.3, IND-SET is NP-Complete. ■

1.5 Vertex Cover

Given a friendship network and an integer k , is there a set of k people such that every friendship includes somebody in the set? Such a set is called a vertex cover. The vertex cover problem is to determine whether a vertex cover of size k exists in the graph.;

Theorem 1.6 *Vertex-Cover is NP-Complete*

Proof: First, show it is in NP. This is trivial, give me a set of k people as a certificate and I can quickly examine every pair of friends and ensure that each friendships consists of someone in the set of k people

Now we show it is NP-complete by reduction from Independent-Set.

Given a graph and an integer k , our goal is to produce a graph G' and integer k' such that G has an ind. set of size k iff G' has a VC of size k' .

This reduction relies on the following lemma:

Lemma 1.7 *A graph $G = (V, E)$ has an independent set of size k iff it has a VC of size $|V| - k$*

Proof: Let I be an ind. set in G of size k . Since I is independent, there is no edge with both endpoints in I , thus every edge must have at least one endpoint in $V \setminus I$, thus $V \setminus I$ is a vertex cover.

Let $V \setminus I$ be a vertex cover of size k . Suppose for $u, v \in I$ there is an edge between u and v (ie. I is not an independent set). Then this is an edge and neither endpoint is in $V \setminus I$, contradicting that $V \setminus I$ is a VC, thus conclude that I is an independent set.

Tldr: If you have an independent set of size k , just take all the other vertices and you have a vertex cover of size $|V| - k$ ■

With this lemma, the reduction becomes trivial. Given a graph G and an integer k , simply return $(G, |V| - k)$, and by the lemma we know that our input has ind. set of size k if and only if the output has a VC of size $|V| - k$, and this reduction is clearly polynomial.

Conclude $\text{IND-SET} \leq_p \text{Vertex-Cover}$, thus by theorem 1.3, Vertex-Cover is NP-Complete ■

1.6 Dominating Set

Intuition: Let graph G represent a social network, say it's a school. And certain members (or vertices) are in a large social group - say a club or a sports team. Is there a social group on campus s.t everyone on campus is either in the club or has a friend in the club?

Problem: Given a graph G , is there a set S of k vertices s.t every vertex in G is either in S or is a neighbor of some vertex in S .

Theorem 1.8 *Dom-Set is NP-Complete*

Proof: First show Dom-Set is in NP. This should be clear, give me a set of k people, I can easily check every person and see if they or a friend of theirs is in the set.

Next show NP-Complete. We do this by reduction from Vertex-Cover.

Given a graph G and an integer k , our goal is to produce a graph G' and integer k' such that G has an ind. set of size k iff G' has a dom. set of size k' .

The reduction proceeds as follows:

Let $G = (V, E)$ be a graph and k be an integer.

Construct a new graph G' and let $G' = G$. For each edge $\langle u, v \rangle$, add a new vertex z and edges such that u, v, z now forms a triangle. Let $k' = k$

**** Draw Examples ****

Return G', k'

Claim 1.9 G has a vertex cover of size k iff G' has a dom set of size k

Proof: Suppose G has a VC S of size k . By definition each edge (u, v) in G has an endpoint in S . Therefore, each triangle (u, z, v) in G' has at least one vertex in S , thus each vertex in G' is in S or has a friend in S .

If G' has a dominating set D of size k . Then, for all $z \in D$, z is either in G or it one of added vertices.

Case 1: z is in G . Then all edges (z, u) that are in G are covered by z . Case 2: z is an added vertex for G' . Then consider the triangle u, z, v that was formed by adding z into our graph. Then, we could remove z and WLOG add u to the Dominating set of $G' \setminus z$, and the edge (u, v) now has one endpoint in DS . By induction, we remove all extraneous edges/vertices and still retain a dom set of size k .

Thus, DS will be a vertex cover. ■

It is clear that this runs in polynomial time, all we do is add at most $2n^2$ edges. ■

1.7 More Cool NP-Complete Problems

1.7.1 Graph Problems

Clique Cover

Hamiltonian Cycle

Traveling Salesperson

Many more...

1.7.2 Optimization Problems

Subset Sum

Partition

Knapsack

Many more...

1.8 For More...

Graph theory: CIS 1600, MATH 3400

Complexity Theory/Algorithms: CIS 2620/5110, CIS 3200/5020

References

- [1] A. WEGNERSON. “Mathematics and Computation”
- [2] S. ARORA and B. BARAK. “Computational Complexity”
- [3] E. TARDOS ET AL. “Algorithm Design”